

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 03

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I Bandaru Jayasri-192424301 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	Conceptual Understanding	Accuracy of Answers	Application of Knowledge	Completeness of Response	Clarity & Presentation	Total
Weightage	30%	20%	15%	20%	15%	
Q1						
Q2						
Q3						

Signature of the student:

B.Jayasri

Total Marks :

Annexure A

Short Answer Test

1. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.
2. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.
3. Write a Python program to implement the **Support Vector Machine (SVM)** algorithm for binary classification. Use a simple dataset and implement a linear kernel. Train the model, classify test instances, and display accuracy. Analyze how the margin and support vectors influence classification performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Naïve Bayes Classifier

Aim

To implement a Naïve Bayes classifier, calculate prior and conditional probabilities, classify a test instance, and compare results with and without Laplace smoothing.

Code

```
# Naive Bayes Classifier

data = [
    ("Sunny", "Hot", "No"),
    ("Sunny", "Hot", "No"),
    ("Overcast", "Hot", "Yes"),
    ("Rainy", "Mild", "Yes"),
    ("Rainy", "Cool", "Yes"),
    ("Sunny", "Cool", "Yes"),
    ("Overcast", "Cool", "Yes"),
    ("Rainy", "Mild", "Yes")
]

test = ("Sunny", "Cool")

classes = ["Yes", "No"]

def predict(smoothing=False):
    scores = {}

    for c in classes:
        count_c = sum(row[2] == c for row in data)
        prior = count_c / len(data)

        prob = prior

        for i, value in enumerate(test):
            count = sum(row[i] == value and row[2] == c for row in data)

            if smoothing:
                total_values = len(set(row[i] for row in data))
                cond = (count + 1) / (count_c + total_values)
            else:
                cond = count / count_c

            prob *= cond

        scores[c] = prob

    return max(scores, key=scores.get), scores

print("Without Laplace Smoothing:")
print(predict(False))

print("\nWith Laplace Smoothing:")
print(predict(True))
```

Concept

Naïve Bayes uses:

$$P(C|X)=P(X)P(X|C)P(C)$$

Since $P(X)$ is common for all classes, classification uses:

$$P(C|X) \propto P(C) \prod_i P(x_i|C)$$

Prior probability:

$$P(C) = \frac{\text{Total samples in } C}{\text{Number of samples in } C}$$

Conditional probability:

$$P(x_i|C) = \frac{\text{Count}(C)}{\text{Count}(x_i, C)}$$

Laplace Smoothing

To avoid zero probability:

$$P(x_i|C) = \frac{\text{Count}(C) + k}{\text{Count}(x_i, C) + 1}$$

where k is the number of possible values.

Analysis

Without smoothing, an unseen feature value can produce probability zero, causing the entire class probability to become zero. Laplace smoothing prevents this problem and makes the classifier more robust.

Conclusion

Naïve Bayes is a simple and efficient probabilistic classifier. Laplace smoothing improves reliability when some feature combinations are missing from the training data.

Q2. Genetic Algorithm

Aim

To implement a simple Genetic Algorithm to maximize a mathematical function using selection, crossover, and mutation.

Code

```
import random

POP_SIZE = 10
GENERATIONS = 20
MUTATION_RATE = 0.1

# Fitness function: maximize x^2
def fitness(x):
```

```

        return x * x

# Initial population
population = [random.randint(0, 31) for _ in range(POP_SIZE)]

for generation in range(GENERATIONS):

    # Selection
    population = sorted(population, key=fitness, reverse=True)
    parents = population[:4]

    # Crossover and mutation
    new_population = parents[:]

    while len(new_population) < POP_SIZE:
        p1, p2 = random.sample(parents, 2)

        # Simple arithmetic crossover
        child = (p1 + p2) // 2

        # Mutation
        if random.random() < MUTATION_RATE:
            child = random.randint(0, 31)

        new_population.append(child)

    population = new_population

    best = max(population, key=fitness)

    print("Generation:", generation + 1,
          "Best:", best,
          "Fitness:", fitness(best))

print("\nBest Solution:", best)
print("Best Fitness:", fitness(best))

```

Working

A Genetic Algorithm contains four main steps:

1. **Initialization** – Create a random population.
2. **Selection** – Select individuals with better fitness.
3. **Crossover** – Combine parents to produce new solutions.
4. **Mutation** – Randomly modify some solutions.

The fitness function used is:

$$f(x)=x^2$$

For values between 0 and 31, the best solution is:

$$x=31$$

and:

f(31)=961

Parameter Analysis

Parameter	Effect
Population size	Larger population gives more diversity
Mutation rate	Higher rate increases exploration
Number of generations	More generations allow more improvement
Selection	Favors better solutions
Crossover	Combines useful solutions

A very low mutation rate may cause premature convergence. A very high mutation rate may make the search unstable.

Conclusion

The Genetic Algorithm gradually improves the population through selection, crossover, and mutation. Proper parameter selection is important for achieving fast convergence and a high-quality solution.

Q3. Support Vector Machine

Aim

To implement a linear SVM for binary classification, classify test instances, calculate accuracy, and understand the effect of margins and support vectors.

Code

```
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Simple binary dataset
X = [
    [1, 1],
    [2, 1],
    [2, 2],
    [3, 2],
    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
]

y = [0, 0, 0, 0, 1, 1, 1, 1]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y, test_size=0.25, random_state=42
)

# Linear SVM
model = svm.SVC(kernel="linear")
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual    :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

# Support vectors
print("Support Vectors:")
print(model.support_vectors_)

```

Working

SVM finds the best separating hyperplane between two classes.

The decision boundary is:

$$wTx+b=0$$

The objective is to maximize the margin between the two classes.

Margin

The margin is the distance between the decision boundary and the closest training points.

A larger margin generally provides better generalization.

Support Vectors

Support vectors are the training points closest to the decision boundary. They determine the position of the optimal separating hyperplane.

If support vectors change, the decision boundary can also change.

Analysis

- A **large margin** generally improves generalization.
- Support vectors are critical to the SVM decision boundary.
- A linear kernel works well when the two classes are approximately linearly separable.
- Incorrect or overlapping data can reduce classification accuracy.

Result

The program displays:

```
Predicted: [...]  
Actual    : [...]  
Accuracy  : 1.0  
Support Vectors:  
[...]  
[...]
```

The exact prediction and support vectors depend on the train-test split.

Conclusion

SVM is an effective binary classification algorithm that finds a separating hyperplane with maximum margin. Support vectors determine the boundary, while a suitable kernel and parameter settings influence classification performance.